Retail: MusicMarkt

COMPUTER SECURITY IN NETWORKS AND SYSTEMS

TAGUSPARK

GROUP 9



António Venâncio 93689
Guilherme Carvalho
Miguel Gonçalves 94238

Business Context

MusicMarkt is an online store that sells all things music related. It offers a large variety of items such as musical instruments, sound systems, CDs, vinyls, and other merchandise.

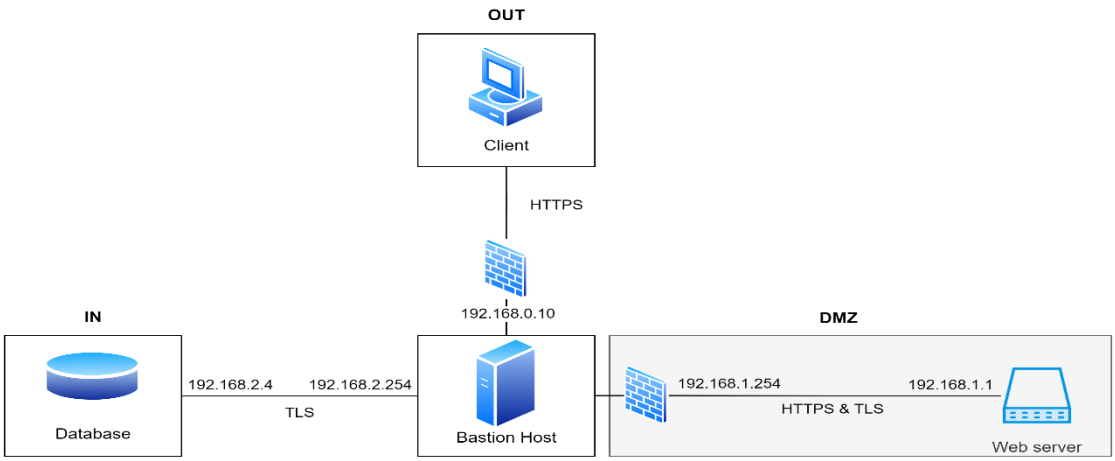
MusicMarkt allows reservations to be made for some new products before they are made available. These products are usually CDs, vinyls, limited edition items, etc.

In order to reward loyal customers and loyal fans, MusicMarkt has a scoreboard for each band and artist that ranks customers by the money they have spent on items from that band or artist. When reservations open for a new item, during the first 24 hours only the top ranked customers in the respective scoreboard(s) can make a reservation.

Infrastructure Overview

Our infrastructure constitutes of four Virtual Machines:

- Client
- Bastion Host
- Webserver (Flask)
- Database (MySQL)



Secure Communications

Our UML above shows which secure channels and protocols we will use. Below is a detailed description of it:

- The Client

  It can only open HTTPS connections (port 43) to the Bastion Host. Note that the client is only aware of the existence of the Bastion Host, i.e., knows the external IP address of it.

- The Bastion Host:

  All incoming HTTPS connections (port 43) are redirected to the Web Server.

  All SSH connections from the external network are redirected to the Database.

  Requests from the internal network 192.168.2.4/24 are only accepted if destined to the SSH port.

  All other traffic is rejected.

- The Web Server:

  Is inside a DMZ.

  Accepts HTTPS connections from both the internal and external networks.

  Accepts SSH connections from the internal network.

  Does not start any new connections.

- The Database:

  Accepts SSH requests.

  Can open SSH connections to both external network and DMZ.

Security Challenge

We intend to tackle the first challenge proposed, which is the following:

"In MusicMarkt, each user has an account that contains personal information to facilitate future orders. The access to the users' accounts must be correctly authenticated using also the user's mobile device."

The requirements are as follows:

R1: Validate authenticity

R2: Avoid compromise of personal information that is stored in the database.

R3: Prevent Brute Force Attacks

R4: Provide connection freshness (both in Login and 2FA Authentication).

R5: Prevent Replay Attacks

Our Proposed Solution

We will be assuming the following:

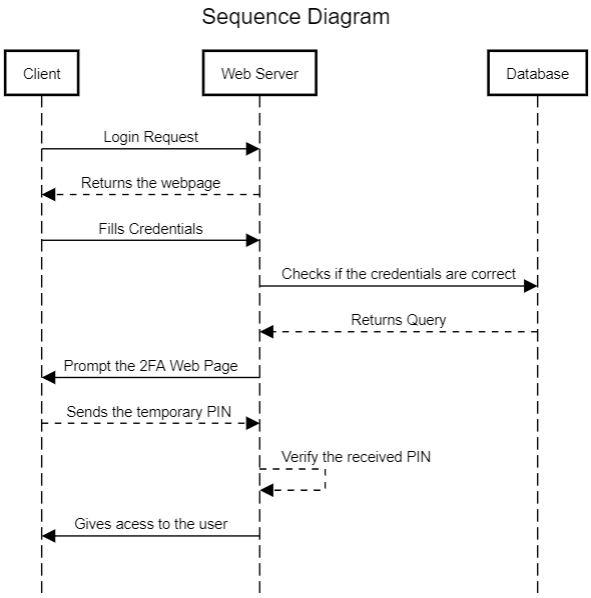
- The Web Server, the Bastion Host and the Database are **partially trustable**.
- The client can be separated into two types:
    - Non authenticated user, which is **not trustable**.
    - Authenticated user, which is **partially trustable**.

The attacker will have two main options to breach security:

- Passing through the firewalls.
- Successfully impersonating a user. This will be the most critical, as it is the most exploitable option. The impersonation can be done in many ways:
    - Getting credentials due to social engineering.
    - Brute Force Attacks
    - Replay Attacks

Our proposed solution to the challenge is as follows:

- The database will contain the username, the password, the cellphone number, and the salt used to hash its password. This is done to ensure safety for the clients in the case of database leaks.
- The initial phase of the authentication is done in the browser, by asking the username and password. To prevent brute force, we'll impose a **limit of attempts** to login with the credentials. If it reaches the limit, the user will be locked out of logging for a certain amount of time.
- After verifying the credentials, to simulate a second device in possession of the user, a 6-digit PIN will be asked. The client must run the "application" that will generate the temporary pin and submit it in the two-factor authentication webpage. The PIN will have **a time limit** to be submitted, otherwise a new one needs to be requested. It will also have a **limit of attempts** to correctly submit the PIN.
- After successfully submitting the PIN, the user will have access to the MusicMarkt's services.



One thing to note here is that we are generating the PIN locally, instead of requesting the mobile operator (which would be **fully trustable**) to send it to the user's number; the latter is the correct option, however we are opting for the former as it simplifies the process.

All certificates and keys were created and signed using OpenSSL, an open-source cross-platform utility used in various digital PKI certificate activities. A self-signed certificate and key were created to represent a Certificate Authority named CA. Two pair of keys (public/private) were generated, one for the Web Server and another for the Database. Both were created and distributed to their respective Virtual Machines.

The correct way would be to obtain a Certificate from an Entity that has the required authority to emit it. However, we used a self-signed certificate, since obtaining a proper certificate requires funds to do so.

To hash the passwords, we used the *hashlib* library. When creating an account, the database stores the hashed password and the salt used to hash it, using **SHA-256**. This way, we can ensure the safety of the user's passwords in the possible cases of database leaks. As mentioned before, this can be extended to other private data of the user.

To run the server, we choose Flask because of the easiness and from previous experience. To host the database, we went with MySQL for compatibility with Flask.

Results

R1: Validate authenticity – **Partially Satisfied**

> We have implemented the 2FA authentication. However, there's an issue that allows a possible attacker to use the cookies to get access to the PIN, and hence the account of the user.

R2: Avoid compromise of personal information that is stored in the database. – **Satisfied**

> We hashed the user's password in the database, preventing the damage caused in the event of a database leak. This can also be extended to other personal information that could be stored, such as credit card number and address.

R3: Prevent Brute Force Attacks –**Satisfied**

> When logging in, if the user fails to login correctly after 3 attempts, the Web Server will lock the user of logging in.

R4: Provide connection freshness (both in Login and 2FA Authentication) – **Satisfied**

> The 2FA Authentication has a time limit of 20 seconds to be successful. In case of failure, the Web Server terminates the session, and another PIN will have to be generated for another Login Attempt. This PIN is not the same between Sessions.

R5: Prevent Replay Attacks– **Satisfied**

> This is done with the combined usage of sessions and time limit to submit the PIN.

Conclusion

With this work, we believe we managed to maintain the 2 most important aspects needed, which are **Freshness** and **Authenticity**, by using appropriate mechanisms for each. Not only that, but we also ensured the protection of data, using hashes, secure channels, and public/private keys. However, we must note that the project is still far from being fully secure and ready for the world.

Moreover, the project can be enhanced in the future. For example:

- On the topic of preventing Brute Force Attacks: it can be improved further upon. Have an exponential lock-out time for logging failures and warning the user of the logging attempts by e-mail/SMS.
- Usage of KEKs and nonces, for the transmission of the PIN.
- Increase the number of Web Servers taking requests. Even though it was out of the scope of our project, for the services of MusicMarkt to escalate, we'd need several Web Server to deal with the requests from many users at a given time. This would also help diminish the effects of a possible DoS attacks.